Name: Vansh Jain
Roll No: 124A8036
Class/Div: G-2
Date of Submission: 15/04/2026

Experiment No.: 10

Title: To implement File Management and I/O Management using File Allocation Strategies (Sequential, Indexed, Linked), Multi-Level Directory Structure, and Disk Scheduling Algorithms (FCFS, SCAN, C-SCAN).

Objective:
Using the CPU-OS simulator analyze and synthesize the following:
a. Understand different file allocation strategies such as sequential, indexed, and linked allocation.
b. Study the concept of file organization using multi-level directory structure.
c. Implement disk scheduling algorithms like FCFS, SCAN, and C-SCAN.
d. Analyze the performance of different disk scheduling techniques based on seek operations.

Software: Ubuntu

Theory: File management is an important function of an operating system that handles the storage, retrieval, and organization of data on storage devices. Files are stored on disk using different allocation methods which determine how disk blocks are assigned to files.
Sequential allocation stores files in contiguous memory locations, making access simple and fast but causing fragmentation issues. Indexed allocation uses an index block to store addresses of all blocks of a file, allowing direct access without fragmentation. Linked allocation stores files as a linked list of disk blocks, where each block points to the next, which avoids fragmentation but may increase access time.A multi-level directory structure organizes files in a hierarchical manner. It allows grouping of files into directories and subdirectories, improving file management, searching, and security. This structure makes it easier to manage large systems with many files and users.Disk scheduling is used by the operating system to decide the order in which disk I/O requests are serviced. Efficient scheduling reduces total seek time and improves system performance. FCFS (First Come First Serve) serves requests in the order they arrive. SCAN (Elevator algorithm) moves the disk arm in one direction servicing requests and then reverses direction. C-SCAN (Circular SCAN) moves in one direction and jumps back to the beginning, providing uniform wait time.These techniques together help in efficient file storage, retrieval, and disk

performance optimization.

Algorithm:

Part A: File Allocation Strategies

1. Start the program.
2. Display menu for Sequential, Indexed, and Linked allocation.
3. Accept user choice.
4. For sequential allocation:
    - Input number of files.
    - Enter starting block and length for each file.
    - Display allocated blocks.
5. For indexed allocation:
    - Input number of files.
    - Enter index block and list of blocks.
    - Display index block and file blocks.
6. For linked allocation:
    - Input number of files.
    - Enter starting block and linked blocks.
    - Display linked sequence.
7. Repeat until user exits.
8. Stop the program.

Part B: Multi-Level Directory Structure

1. Start the program.
2. Input number of directories.
3. Enter parent and child directory names.
4. Store the directory structure.
5. Display all directory relationships.
6. Accept a directory name to search.
7. Display all subdirectories under the given directory.
8. Stop the program.

Part C: Disk Scheduling Algorithms

FCFS Algorithm

1. Start the program.
2. Input number of disk requests.
3. Enter request sequence.
4. Enter initial head position.
5. Process requests in given order.
6. Calculate total seek operations.
7. Display seek sequence and total seek time.

SCAN Algorithm

1. Sort the disk requests.
2. Move the disk head in one direction servicing requests.
3. When end is reached, reverse direction.
4. Continue servicing remaining requests.
5. Calculate total seek operations.
6. Display results.

C-SCAN Algorithm

1. Sort the disk requests.
2. Move the head in one direction servicing requests.
3. After reaching the end, jump to the beginning.
4. Continue servicing remaining requests.
5. Calculate total seek operations.
6. Display results.

Output/Result:

Part A:

```
siesgst@aids: ~/naitik/oslab

siesgst@aids:~$ cd ~
siesgst@aids:~$ cd naitik
siesgst@aids:~/naitik$ cd oslab
siesgst@aids:~/naitik/oslab$ vi file_allocation.c
siesgst@aids:~/naitik/oslab$ gcc file_allocation.c -o file_allocation
siesgst@aids:~/naitik/oslab$ ./file_allocation


===== File Allocation Menu =====
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Exit
Enter your choice: 1

--- Sequential File Allocation ---
Enter number of files: 2

Enter starting block and length of file 1: 10 4
File 1 allocated blocks: 10 11 12 13

Enter starting block and length of file 2: 20 3
File 2 allocated blocks: 20 21 22


===== File Allocation Menu =====
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Exit
Enter your choice: 2

--- Indexed File Allocation ---
Enter number of files: 1

Enter index block of file 1: 5
Enter number of blocks allocated: 4
Enter the blocks: 11 13 17 19
File 1 -> Index Block: 5
Allocated blocks: 11 13 17 19
```

siesgst@aids: ~/naitik/oslab

```
===== File Allocation Menu =====
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Exit
Enter your choice: 2

--- Indexed File Allocation ---
Enter number of files: 1

Enter index block of file 1: 5
Enter number of blocks allocated: 4
Enter the blocks: 11 13 17 19
File 1 -> Index Block: 5
Allocated blocks: 11 13 17 19


===== File Allocation Menu =====
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Exit
Enter your choice: 3

--- Linked File Allocation ---
Enter number of files: 1

Enter starting block of file 1: 7
Enter number of blocks: 4
Enter the linked blocks: 9 14 18 22
File 1 linked allocation sequence: 7 -> 9 -> 14 -> 18 -> 22


===== File Allocation Menu =====
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Exit
Enter your choice: 4
Exiting...
siesgst@aids:~/naitik/oslab$
```

siesgst@aids: ~/harini/oslab

```c
#include <stdio.h>
#include <stdlib.h>

void sequentialAllocation() {
    int n, start, i, length;

    printf("\n--- Sequential File Allocation ---\n");
    printf("Enter number of files: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nEnter starting block and length of file %d: ", i + 1);
        scanf("%d%d", &start, &length);

        printf("File %d allocated blocks: ", i + 1);
        for(int j = start; j < start + length; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }
}

void indexedAllocation() {
    int n, indexBlock, blocks[20], count, i, j;

    printf("\n--- Indexed File Allocation ---\n");
    printf("Enter number of files: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nEnter index block of file %d: ", i + 1);
        scanf("%d", &indexBlock);

        printf("Enter number of blocks allocated: ");
        scanf("%d", &count);

        printf("Enter the blocks: ");
        for(j = 0; j < count; j++) {
            scanf("%d", &blocks[j]);
        }
```

siesgst@aids: ~/harini/oslab

```
        for(j = 0; j < count; j++) {
            scanf("%d", &blocks[j]);
        }

        printf("File %d -> Index Block: %d\n", i + 1, indexBlock);
        printf("Allocated blocks: ");
        for(j = 0; j < count; j++) {
            printf("%d ", blocks[j]);
        }
        printf("\n");
    }
}

void linkedAllocation() {
    int n, start, count, blocks[20], i, j;

    printf("\n--- Linked File Allocation ---\n");
    printf("Enter number of files: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nEnter starting block of file %d: ", i + 1);
        scanf("%d", &start);

        printf("Enter number of blocks: ");
        scanf("%d", &count);

        printf("Enter the linked blocks: ");
        for(j = 0; j < count; j++) {
            scanf("%d", &blocks[j]);
        }

        printf("File %d linked allocation sequence: %d", i + 1, start);
        for(j = 0; j < count; j++) {
            printf(" -> %d", blocks[j]);
        }
        printf("\n");
    }
}
```

siesgst@aids: ~/harini/osl

```
        printf("File %d linked allocation sequence: %d", i + 1, start);
        for(j = 0; j < count; j++) {
            printf(" -> %d", blocks[j]);
        }
        printf("\n");
    }
}

int main() {
    int choice;

    do {
        printf("\n\n===== File Allocation Menu =====\n");
        printf("1. Sequential Allocation\n");
        printf("2. Indexed Allocation\n");
        printf("3. Linked Allocation\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                sequentialAllocation();
                break;
            case 2:
                indexedAllocation();
                break;
            case 3:
                linkedAllocation();
                break;
            case 4:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice!\n");
        }
    } while(choice != 4);

    return 0;
}
```

Part B:

```
siesgst@aids:~/naitik/oslab$ vi multi_directory.c
siesgst@aids:~/naitik/oslab$ gcc multi_directory.c -o multi_directory
siesgst@aids:~/naitik/oslab$ ./multi_directory

--- Multi-Level Directory Structure ---
Enter number of directories: 4

Enter parent directory of directory 1: root
Enter child directory of directory 1: home

Enter parent directory of directory 2: home
Enter child directory of directory 2: user

Enter parent directory of directory 3: user
Enter child directory of directory 3: documents

Enter parent directory of directory 4: user
Enter child directory of directory 4: downloads

Directory Structure:
root -> home
home -> user
user -> documents
user -> downloads

Enter parent directory name to search its child directories: user
Child directories under user are:
documents
downloads
siesgst@aids:~/naitik/oslab$
```

siesgst@aids: ~/harini/oslab

```c
#include <stdio.h>
#include <string.h>

struct Directory {
    char parent[30];
    char child[30];
};

int main() {
    struct Directory dir[20];
    int n, i;
    char search[30];
    int found = 0;

    printf("\n--- Multi-Level Directory Structure ---\n");
    printf("Enter number of directories: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nEnter parent directory of directory %d: ", i + 1);
        scanf("%s", dir[i].parent);

        printf("Enter child directory of directory %d: ", i + 1);
        scanf("%s", dir[i].child);
    }

    printf("\nDirectory Structure:\n");
    for(i = 0; i < n; i++) {
        printf("%s -> %s\n", dir[i].parent, dir[i].child);
    }

    printf("\nEnter parent directory name to search its child directories: ");
    scanf("%s", search);

    printf("Child directories under %s are:\n", search);
    for(i = 0; i < n; i++) {
        if(strcmp(dir[i].parent, search) == 0) {
            printf("%s\n", dir[i].child);
            found = 1;
        }


    if(!found) {
        printf("No child directory found under %s\n", search);
    }

    return 0;
}
```

Part C:

```
siesgst@aids:~/naitik/oslab$ vi disk_scheduling.c
siesgst@aids:~/naitik/oslab$ gcc disk_scheduling.c -o disk_scheduling
siesgst@aids:~/naitik/oslab$ ./disk_scheduling
Enter number of disk requests: 8
Enter disk request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter total disk size: 200

--- FCFS Disk Scheduling ---
Seek Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Seek Operations = 640

--- SCAN Disk Scheduling ---
Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total Seek Operations = 331

--- C-SCAN Disk Scheduling ---
Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37
Total Seek Operations = 382
siesgst@aids:~/naitik/oslab$
```

siesgst@aids: ~/harini/osl

```c
#include <stdio.h>
#include <stdlib.h>

void sort(int arr[], int n) {
    int i, j, temp;
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void fcfs(int req[], int n, int head) {
    int i, seek = 0;

    printf("\n--- FCFS Disk Scheduling ---\n");
    printf("Seek Sequence: %d ", head);

    for(i = 0; i < n; i++) {
        seek += abs(req[i] - head);
        head = req[i];
        printf("-> %d ", head);
    }

    printf("\nTotal Seek Operations = %d\n", seek);
}

void scan(int req[], int n, int head, int disk_size) {
    int seek = 0, i, pos;
    int temp[20];

    for(i = 0; i < n; i++)
        temp[i] = req[i];

    sort(temp, n);
```

siesgst@aids: ~/harini/oslab

```
void scan(int req[], int n, int head, int disk_size) {
    int seek = 0, i, pos;
    int temp[20];

    for(i = 0; i < n; i++)
        temp[i] = req[i];

    sort(temp, n);

    for(i = 0; i < n; i++) {
        if(temp[i] >= head) {
            pos = i;
            break;
        }
    }

    printf("\n--- SCAN Disk Scheduling ---\n");
    printf("Seek Sequence: %d ", head);

    for(i = pos; i < n; i++) {
        seek += abs(temp[i] - head);
        head = temp[i];
        printf("-> %d ", head);
    }

    seek += abs((disk_size - 1) - head);
    head = disk_size - 1;
    printf("-> %d ", head);

    for(i = pos - 1; i >= 0; i--) {
        seek += abs(temp[i] - head);
        head = temp[i];
        printf("-> %d ", head);
    }

    printf("\nTotal Seek Operations = %d\n", seek);
}

void cscan(int req[], int n, int head, int disk_size) {
```

siesgst@aids: ~/harini/oslab

```c
void cscan(int req[], int n, int head, int disk_size) {
    int seek = 0, i, pos;
    int temp[20];

    for(i = 0; i < n; i++)
        temp[i] = req[i];

    sort(temp, n);

    for(i = 0; i < n; i++) {
        if(temp[i] >= head) {
            pos = i;
            break;
        }
    }

    printf("\n--- C-SCAN Disk Scheduling ---\n");
    printf("Seek Sequence: %d ", head);

    for(i = pos; i < n; i++) {
        seek += abs(temp[i] - head);
        head = temp[i];
        printf("-> %d ", head);
    }

    seek += abs((disk_size - 1) - head);
    head = disk_size - 1;
    printf("-> %d ", head);

    seek += abs(head - 0);
    head = 0;
    printf("-> %d ", head);

    for(i = 0; i < pos; i++) {
        seek += abs(temp[i] - head);
        head = temp[i];
        printf("-> %d ", head);
    }
```

siesgst@aids: ~/harini/oslab

```
    seek += abs((disk_size - 1) - head);
    head = disk_size - 1;
    printf("-> %d ", head);

    seek += abs(head - 0);
    head = 0;
    printf("-> %d ", head);

    for(i = 0; i < pos; i++) {
        seek += abs(temp[i] - head);
        head = temp[i];
        printf("-> %d ", head);
    }

    printf("\nTotal Seek Operations = %d\n", seek);
}

int main() {
    int req[20], n, head, disk_size, i;

    printf("Enter number of disk requests: ");
    scanf("%d", &n);

    printf("Enter disk request sequence: ");
    for(i = 0; i < n; i++) {
        scanf("%d", &req[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter total disk size: ");
    scanf("%d", &disk_size);

    fcfs(req, n, head);
    scan(req, n, head, disk_size);
    cscan(req, n, head, disk_size);

    return 0;
}
```

Conclusion: The experiment successfully demonstrated File Management and I/O Management techniques in an operating system. Different file allocation methods such as sequential, indexed, and linked allocation were implemented and analyzed. Multi-level directory structure was used to organize files efficiently in a hierarchical manner. Disk scheduling algorithms FCFS, SCAN, and C-SCAN were also implemented to study their performance in terms of seek operations. These techniques help in improving storage efficiency, file organization, and overall system performance.